Dt: 03/09/26

UNIT-03 : Graph & Tree Algorithms :

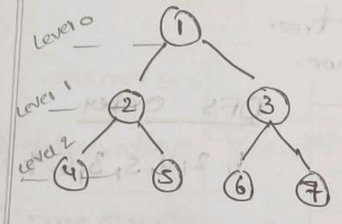
Traversal Algorithms: DFS & BFS

Breadth First Search

- Breadth First Search is a graph traversal Algorithm that starts from Source node & explore the graph level
- first it Visit all nodes directly adjacent to Source

Nodes which are having direct connected to Source node

- Then Traverse the adjacent node till end of the list
- It Uses queue data structure.



~~1~~	~~2~~	~~3~~	~~4~~	~~5~~	~~6~~	~~7~~

BFS sequence

1, 2, 3, 4, 5, 6, 7

తెలుగు అర్థము.

→ 1 కి 2, 3 adjacent nodes ఉన్నాయి
→ సో ముందు 1 నీ కింద sequence లో read చేశాం.
→ ఇలా 2 adj లో 4, 5 Q లో రాశాం.
→ 2 ని read చేశాక
→ 3 కి 6, 7 Q లో
→ 3 ని read చేశాక 4, 5, 6, 7 కి ఇలా చేసి రాస్తాం.

BFS (root)

```
1) If root is ~~NULL~~ Nil, return
2) Create an empty Q
3) Insert root into Q
4) While Q is not empty
    a) Remove a node from front of Q
    b) Visit the removed node
    c) If the left child exists
        - Insert left child into Q.
```

d) If the right child ~~left~~ exists

— Insert right child into Q

5) Continue the Process ~~tr~~ till end of the list

6) Stop

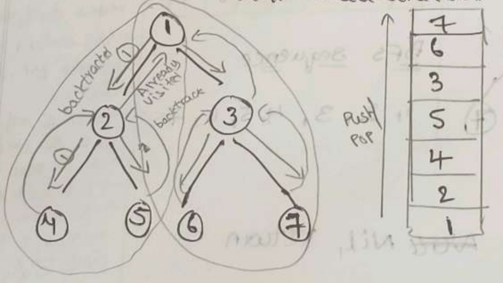
Depth-first Search (DFS) Algorithm

— In depth first Search for a graph adjacent Vertices are traversed one by one

— When we traverse the Vertices the traverse it 1 from start to end in a Particular Sequence.

— This is Similar to Pre Order binary tree traversal where you have to traverse the first tree & then 2nd tree.

— works on stack Data structure.



DFS Order

1, 2, 4, 5, 3, 6, 7

Popped.

తెలుగు తర్జుమా :

→ ముందు 1 ని stack లో వెయ్యాలి

→ కింద ఉన్న 2, దాని కింద ఉన్న 4 ని కూడా stack లో వెయ్యాలి

→ 4 కి కింద ఏమి లేవు కాబట్టి backtrack to 2, 2 కి 5 ఉంది దానిని stack లో Put

→ 5 కి ఏమి లేవు backtrack to 2, 2 already written కాబట్టి backtrack to 1, మళ్లీ 1 already ఉంది కాబట్టి, 3 కి వెళ్ళు

DFS -

1) If

2) Vis

3) Ad

4) A

5) -

left

6) -

7) -

&

8)

Pa

1) A

2) D

3)

DFS Algorithm :

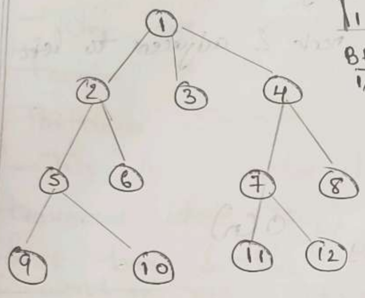
1) If root is NULL, return
2) Visit the root Node
3) Add node in~~to~~ the stack
4) Apply Preorder to left Subtree
5) Traverse the left node & adjacent to left & add to stack
6) Apply Preorder to right Subtree
7) Traverse the left node & adjacent to left & add to stack
8) Stop.

Time Complexity
Space Complexity } O(n)

Parameters	BFS	DFS
1) Abrevations	Breadth first Search	Depth first Search
2) Data structure Used	Queue	Stack.
3) Def	BFS is traversal approach in which we 1st visit all nodes on the same level before moving to the Next level	DFS is traversing Approach in which we traverse within the root-Node & procee through Node which is adjacent to Previo Node.

4) Conceptual difference	Builds tree level by level	Builds tree by Subtree & Depthwise
5) Approach used	uses FIFO Approach	uses LIFO Approach
6) App's	Bi-partite Graph & shortest Path	Acyclic Graph & Topological Sorting

Example ①



BFS

1	2	3	4	5	6	7	8	9	10	11	12

BFS order
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12

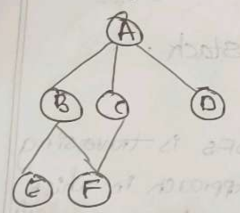
DFS
4 7 11 12 8

3
6
10
9
5
2
1

DFS Sequence

1, 2, 5, 9, 10, 6, 3, 4, 7, 11, 12, 8

Example ②



BFS

A	B	C	D	E	F

from B → E, F
C → F → already visited.

order

A B C D E F

DFS

D
C
F
E
B
A

DFS Sequence

A, B, E, F, C, D

Dt: 04/09/26

Shortest Path Algorithms

- Shortest Path ~~Algorithm~~ Problem is a Problem of finding the shortest Path (s) between Vertex of a given graph.

Types

1) Two-Pairs Shortest Path
2) Single Source Shortest Path
3) Single destination Shortest Path
4) All-Pair Shortest Path.

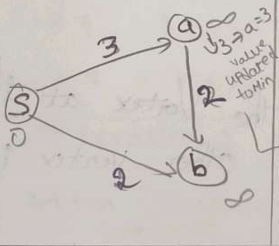
(*) Single - Source Shortest Path

Bellman ford Algorithm

- Beuman ford is a Single Source shortest Path algorithm used to find the shortest Path
- It is Used in Computer Network
- This Algorithm found by Richard Bellman & Loster ford
- The Beuman-ford also useful when graph Carries- Negative weights.

Edge Relaxation:

Consider the edge (a,b) in the graph.



d[S] = 0

1) d[a] = d[S] + w
= 0 + 3
min (∞, 3)
d[a] = 3

2) d[b] = d[a] + w
= 3 + 2 = 5
d[b] = d[a] + w
= 0 + 2 = 2
min (5, 2)
d[b] = 2

If $\underset{\text{distance}}{d[a]} + \underset{\text{weight}}{w} < d[b]$ then $d[b] = d[a]+w$

Algorithm

1) Start with a Source Vertex

2) Set distance to Source Vertex as 0 & rest Nodes as ∞

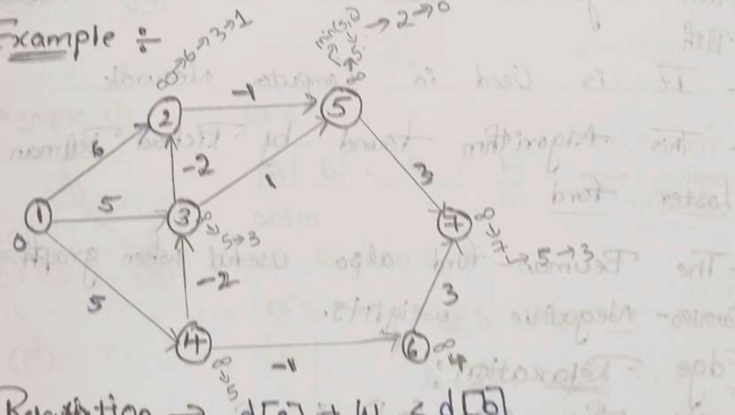
3) for each edge, check if the distance to the destination can be made shorter by going through another Vertex.

4) Repeat this Process for all the Vertex(V-1) iterations where 'V' is the vertex number.

5) Finally check the Negative Weight cycles If any distance can still reduce

6) Stop.

Example :



Relaxation → $d[a] + w < d[b]$
then $d[b] = d[a] + w$

Step 1 : Initialization

Initially the distance to the Vertex at the Source (1) is set 0, all other vertex is Set to ∞ initially.

Vertex	1	2	3	4	5	6	7
Distance	0	∞	∞	∞	∞	∞	∞

Step 2 : Relaxation

There are 7 Vertices, So in Step 2, we relax $(7-1) = 6$ edges 6 times.

Iteration : 1

$1 \to 2 = 0+6 = 6$

$1 \to 3 = 0+5 = 5$

$1 \to 4 = 0+5 = 5$

$2 \to 5 = 6+(-1) = 5$

$3 \to 2 = 5+(-2) = 3$ $\min(6,3) = 3$

$3 \to 5 = 5+1 = 6$ $\min(5,6) = 5$

$4 \to 3 = 5+(-2) = 3$ $\min(5,3) = 3$

$4 \to 6 = 5+(-1) = 4$

$5 \to 7 = 5+3 = 8$

$6 \to 7 = 4+3 = 7$ $\min(7,8) \Rightarrow 7$

Iteration 1	1	2	3	4	5	6	7
After Iteration weight 1 distance	0	3	3	5	5	4	7

Iteration : 2

Dt : 09/08/26

$1 \to 2 = 0+6 = 6, \min(6,3) \Rightarrow 3$

$1 \to 3 = 0+5 = 5$ $\min(5,3) \Rightarrow 3$

$1 \to 4 = 0+5 = 5$ No change (N C)

$2 \to 5 = 3+(-1) = 2, \min(5,2) = 2$

$3 \to 2 = 3+(-2) = 1$ $\min(3,1) = 1$ 0 1 3 5 2 4 5

$3 \to 5 = 3+1 = 4$ $\min(2,4) = 2 \to$ N.C

$4 \to 3 = 5+(-2) = 3$ N.C

$4 \to 6 = 5+(-1) = 4$ N.C

$5 \to 7 = 2+3 = 5$ $\min(5,6) = 5$

$6 \to 7 = 4+3 = 7$ $\min(5,7) = 5$

Iteration 2	1	2	3	4	5	6	7
After iteration weight	0	1	3	5	2	4	5

Iteration 3

$1 \rightarrow 2 = 0+6$ $\min(1, 6) =$ ①

$1 \rightarrow 3 = 0+5 = 5$ $\min(5, 3) =$ ③

$1 \rightarrow 4 = 0+5 = 5$ N.C

$2 \rightarrow 5 = 1+(-1) = 0$ $\min(2, 0) = 0$

$3 \rightarrow 2 = 3+(-2) = 1$ N.C

$3 \rightarrow 5 = 3+1 = 4$ $\min(4, 0) = 0$

$4 \rightarrow 3 = 5+(-2) = 3$ N.C

$4 \rightarrow 6 = 5+(-1) = 4$ N.C

$5 \rightarrow 7 = 0+3 = 3$ $\min(5, 3) = 3$

$6 \rightarrow 7 = 4+3 = 7$ $\min(7, 3) =$ ③

Iteration 3	1	2	3	4	5	6	7
After iteration wt	0	1	3	5	0	4	3

Iteration 4

$1 \rightarrow 2 = 0+6 = 6$ $\min(6, 1) = 1$

$1 \rightarrow 3 = 0+5 = 5$ $\min(5, 3) = 3$

$1 \rightarrow 4 = 0+5 = 5$ N.C

$2 \rightarrow 5 = 1+(-1) = 0$ N.C

$3 \rightarrow 2 = 3+(-2) = 1$ N.C

$3 \rightarrow 5 = 3+1 = 4$ $\min(5, 0) = 0$ N.C

$4 \rightarrow 3 = 5+(-2) = 3$ N.C

$4 \rightarrow 6 = 5+(-1) = 4$ N.C

$5 \rightarrow 7 = 0+3 = 3$ N.C

$6 \rightarrow 7 = 4+3 \Rightarrow 7$, $\min(3, 7) = 3$ N.C

Iteration 4	1	2	3	4	5	6	7
After Iteration weight	0	1	3	5	0	4	3

→ Now iteration 3 & iteration 4 have the same answers & 4 is stopped changing

→ If we apply the iteration 5 & 6 According to the V-1 the Answer is same as iteration 4, 5, 6

→ Hence the shortest Distance from starting node to all other node is

Iteration 4, 5, 6	1	2	3	4	5	6	7
~~After~~ Iteration	0	1	3	5	0	4	3

Time Complexity :

Given a graph with V vertices & 'E' edges.

- It performs edge investigation V-1 times.
- checking all edges one takes E time

$$\text{Total Time} = (\overset{\text{No. of iteration}}{V-1}) \times \overset{\text{10 edges in previous EX}}{E} \rightarrow \text{Total no. of edges}$$

$$\boxed{= O(V \times E)}$$

→ [illegible].

$$\text{Space Complexity} = \boxed{O(V)}$$

(*) Floyd - Warshall Algorithm :

→ Floyd - Warshall Algorithm is used to solve all pair shortest path problem.

→ It computes shortest path between every pair of vertices of the given graph.

→ It is the example of dynamic Programming Approach

ಋಣ -ve values

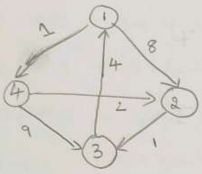
→ No Negative cycles are there.

— There are distances matrix needs to be created as per & equal to Number of Vertices.

If graph = 5 vertex

So, total distance matrix will be 5, of 5×5 order.

Dt:- 11/09/26



Using the Floyd Warshall-Algorithm. Find the shortest Path distance b/w every pair of Vertices.

Solution

[illegible] vertex [illegible] edges [illegible] Remove [illegible]

1) Step 1:- Remove all the self loops and Parallel edges (keeping the lowest weight edge from the graph)

2) In the given graph there are neither self edges nor Parallel edges

3) There are 4 Vertices in the graph so there will 4 matrices of Order 4×4 in the soln.

Step 2:-

- Write the initial distance matrix
- It represents the distance from every Pair of Vertices in the form of given weights
- for diagonal elements, distance Value = 0
- Vertices with direct edge, $\boxed{\text{distance Value} = \text{Weight}}$
- Vertices with no direct edge

$\boxed{\text{distance Value} = \infty}$

The Initial distance matrix of the graph is

(Input Matrix [illegible] (Vertex) Matrices [illegible])

Input ↓ D_0 ↓ Distance Matrix

$$D_0 = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 8 & \infty & 1 \\ 2 & \infty & 0 & 1 & \infty \\ 3 & 4 & \infty & 0 & \infty \\ 4 & \infty & 2 & 9 & 0 \end{array}$$

[1 [illegible] 2 [illegible]

[[illegible] 0 [illegible]]

Step 3: Using Floyd-Warshall Algorithm Calculate the distance matrix 1

$$D_1 = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 8 & \infty & 1 \\ 2 & \infty & 0 & 1 & \infty \\ 3 & 4 & \boxed{12} & 0 & \boxed{5} \\ 4 & \infty & 2 & 9 & 0 \end{array}$$

1) 1st row/1column of D_0 → copy [illegible]
2) Diagonal Value = 0
3) If ∞ is present copy the value from D_0 (↳ 1st row [illegible] / 1st column [illegible])
4) $8+4=12$, $4+1=5$ } → equivalent to particular row/column

Step 4: Using Floyd-Warshall Algo- calculate the distance Matrix 2 =

$$D_2 = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 8 & \boxed{9} & 1 \\ 2 & \infty & 0 & 1 & \infty \\ 3 & 4 & 12 & 0 & 5 \\ 4 & \infty & 2 & \boxed{3} & 0 \end{array}$$

1) consider 2 row/2nd column (→ D_2 [illegible])
2) Put diagonal element 0
3) If ∞ is present copy the value from D_1 (2nd row/2nd column)
4) $8+9=9$ $\min(\infty,9)=9$
$2+1=3$ $\min(3,9)=3$.

Step 5: Using Floyd-Warshall Algorithm calculate the distance Matrix 3

$$D_3 = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 8 & 9 & 1 \\ 2 & 5 & 0 & 1 & 6 \\ 3 & 4 & 12 & 0 & 5 \\ 4 & 7 & 2 & 3 & 0 \end{array}$$

1) Consider 3 row/3 column from D_2
2) Put Diagonal elements = 0
3) $4+1$ $\min(\infty,5)=5$
(i) $\underset{=7}{4+5}$ $\min(\infty,7)=7$
(i) $9+12 \Rightarrow 21$ $\min(8,21)=8$
(i) $12+3=15$ $\min(2,15)=2$

$9+5=14$ $\min(1,14) = 1$
$5+1=6$ min =

Step 6 ÷ Using Floyd Warshall Algorithm calculate distance Matrix 4.

$$D4 = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 3 & 4 & 1 \\ 2 & 5 & 0 & 1 & 6 \\ 3 & 4 & 7 & 0 & 5 \\ 4 & 7 & 2 & 3 & 0 \end{array}$$

1) Consider 4 row/4 column
2) Put diagonal element 0
3) $7+6=13$ $\min(5,13)=5$
   $7+5=12$ $\min(4,12)=4$
   $2+1=3$ $\min(8,3)=3$
   $5+7=7$ $\min(12,7)=7$
   $1+3=4$ $\min(9,4)=4$
   $6+3=9$ $\min(1,9)=1$

Time Complexity :
Space Complexity : } O(n3)

Algorithm

Dt ÷ 17/09/24

1) Create n×n matrix to hold the shortest distance

2) Initialize the matrix for every Pair of Vertices i & j

Set d[i][j] = W[i][j], → Put 0, when i equals to j, (diagonal elements)

→ Put edge value, when edge from i to j is exists

→ Put ∞ when no edge exists

3) Step 3 : Repeat Step 4 to 6 for k=1 to n times 'k' → is the Vertex which is newly Permitted to create intermediate Path.

4) **Step 4** ÷ for recurrent Value of k, repeat step 5 ~~&6~~, for every Source Vertex to i=1 to n

5) Step 5 ÷ Compare cost of path already know d[i][j]

with the path which is already discovered through Vortex K

$d[i][k] + d[k][j]$

6) **step6:-** After all n values of 'k' have been Processed, the matrix 'd' adds the shortest distance of every Pair of Vertices return to d.

(*) String Matching Algorithms:-

1) Navie String Matching Algorithm:-

- It is the Simplest method for pattern - matching
- performs check at all the Positions in the text, whether an Occurence of the Pattern is started (or) not
- After each attempt algorithm shifts pattern exactly Once.

(*) **No. of Windows Calculations:-**

→shift 0 window, equal గా తీసుకోవాలి → string.
→shift 1 window

Example:- T = a a b a a c a a d a a b a a b a

(i) index → 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Input string.

Pattern ⇒ P = a a b a

(j) 0 1 2 3

$T = n = 16$

$P = m = 4$

$$\boxed{\text{No. of Windows} = n - m + 1}$$

$= 16 - 4 + 1$

$= 12 + 1 \Rightarrow 13$

So Shift $S = 0$ to 12

aaba

Shift (s)	Window	Comparison Sequence	Comparison used	Result ~~match at 0~~
0	aaba	a=a, a=a, b=b, a=a	4	match at 0 index → (s)
1	abaa	a=a, b≠a, b≠a, a=a	2	fail at j=1
2	baac	a≠b, a=a, b≠a, a≠c	1	fail at j=0
3	aaca	a=a, a=a, b≠a, a=a	3	fail at j=2
4	acaa	a=a, c≠a, b≠a, a=a	2	fail at j=1
5	caad	a≠c, a=a, b≠a d≠a	1	fail at j=0
6.	aada	a=a, a=a, b≠d a=a	3	fail at j=2
7	adaa	a=a, a≠d, b≠a, a=a	2	fail at j=1
8.	daab	a≠d, a=a, b≠a a≠b	1	fail at j=0
9.	aaba	a=a, a=a, b=b a=a	4	Match at 9 → (i)
10	abaa	a=a, b≠a, b≠a, a=a	2	fail at j=1.
11	baab	a≠b, a=a, b≠a a≠b	1	fail j=0
12	aaba	a=a, a=a, b=b, a=a	4	match at 12

Output :- The expected Outcome of the string

T = aabaa caa daa baa ba

P = aaba is at index : 0, 9 & 12

Time Complexity : O(n)

Space Complexity : O(1)

Algo :

```
NAYIE-STRING-MATCH (T, P)
    n = length (T)
    m = length (P)
for S=0 to n-m
if P[1....m] = T = [S+1... S+m]
Printf ("Pattern occurs with shift");
        n = length (T)
        m = length (P)
      for i=0 to n-m
          j=0
      while j<m & T[i+j] = P[j]
            j = j+1
          if j==m
            Print = j
```

ఆ pattern ని ఎంత వరకు చెక్ చేస్తున్నాం ఇది

T[i+j] → Comparison sequen

P[j] → pattern

Dt: 23/09/24.

(*) Knuth - Morries Pratt (KMP) Algorithm :

→ It is used to solve the pattern matching algorithm, which is the task of finding all occurences of a given Pattern

LPS = Longest Proper Prefix → π (Pie) → Table

దీనిని కూడా అంటారు

Pattern	LPS Array	
$\overset{0}{A}\overset{1}{A}\overset{2}{A}\overset{3}{A}$	0123	→ 1st time వస్తే 0 పెట్టాలి 2nd time వస్తే 1 పెట్టాలి.
ABCDEA	000001	→ ఇక్కడ చూస్తే 1st time ye వచ్చాయి కాబట్టి 0 పెట్టాం.
AABCDABEABF	0000120120	→ ~~ఇక్కడ A కి index 0, 2nd time వచ్చింది 1, [illegible] B, 2nd time వచ్చింది 2 పెట్టి~~

Example :-

(→ initial)

A B A B C A B C A B A B A B D (LPS 2)

(i → next)

Pattern :- A B A B D

Soln :-

1st time [illegible]

Calculate LPS Array.

Multiple occur [illegible] A 8 index position 1, for B index position is 2.

j	A	B	A	B	D	
0	0	0	1	2	0	← LPS
	1	2	3	4	5	← index.

String	Pattern (Match [illegible])	Result
A → i	$\overset{j}{0}+1 \rightarrow A$	Match i = j
B → i	$\overset{j}{1}+1 \rightarrow B$	Match i = j
A → i	$2+1 \rightarrow A$	Match i = j
B → i	$3+1 \rightarrow B$	Match i = j
C → i	$4+1 \rightarrow D$	Not match $C \neq D$.

If match i = j increment i & increment j ↓ Not match index position [illegible] LPS value (2) index [illegible] B at index [illegible] Not [illegible] LPS (0) → [illegible] index [illegible]

Now if there is a mismatch shift index of j to the LPS Position as Per LPS table.

C → i $\overset{j}{2}+1 \rightarrow 3$ Not match $C \neq A$

Again shift index of j to the LPS Position as Per LPS table (Shifted to 0)

C → i $\overset{j}{0}+1 = A$ Not match $C \neq A$

So we can't move '0' further to left so shift i to A   (Explanation pencil [illegible])

A → i $\overset{j}{0}+1 = A$ match i = j

B → i $\overset{j}{1}+1 = B$ match i = j

C → i $\overset{j}{2}+1 = A$ Not match $C \neq A$

As there is mismatch shift index of j to LPS

Shifted

C → i

Shift

A →

B →

A

B

A

Mism

to

Shifted to '0' as H was at index 2.

C → i 0+1 = A Not match $C \neq A$

Shift again to left & increment C to A, i → A

A → i $\overset{j}{0}+1 = A$ match i=j
B → i $\overset{j}{1}+1 = B$ match i=j
A → i $\overset{j}{2}+1 = A$ match i=j
B → i $\overset{j}{3}+1 = B$ match i=j
A → i $\overset{j}{4}+1 = D$ Not match $A \neq D$

i [illegible] c [illegible] j [illegible] shift [illegible]. match [illegible] j [illegible] shift [illegible] zero [illegible] c [illegible] 5 [illegible], c [illegible] increment [illegible] i → A [illegible]

Mismatch is there hence shift position of j to the LPS position (shift to 2 LPS)

A → i $\overset{j}{2}+1 = A$ match i=j
B → i $\overset{j}{3}+1 = B$ match i=j
D → i $\overset{j}{4}+1 = D$ match i=j

string End [illegible] [illegible].

End of String & End of Pattern, Hence String matching Pattern is ABABD at index 10.

Dt:- 30/09/26

(*) Rabin-Karp Algorithm:

- It is used to search (or) match the pattern in the given text using a hash fn.

"A hash fn is a tool to map a larger input value to a smaller output value. This output value is called as "Hash value"

1) Weak Hash (sum of codes) weak Hash ని ఇలా కూడా [illegible].

a = 1, b = 2, c = 3, d = 4, e = 5

Soon

Hash. Sum of Codes of all characters.

Example :-

T = cc a cc a a e d b a (n = 11)

P = d b a (m = 3)
↳ Pattern.

Sol

Hash Value of Pattern Using Sum of codes is 4+2+1

dba = 4+2+1
= 7

(ఒక మాటలో చెప్పాలి అంటే
Spurious → value మ్యాచ్ అవుతుంది కానీ Pattern మ్యాచ్ అవ్వదు.

Shift	Window	Sum	Result.
0	cca	3+3+1 = 7.	Spurious
1	cac	3+1+3 = 7	Spurious.
2	acc	1+3+3 = 7	Spurious
3	cca	3+3+1 = 7	Spurious
4	caa	3+1+1 = 5	
5	aae	1+1+5 = 7	Spurious
6	aed	1+5+4 = 10	
7	edb	5+4+2 = 11	
8	dba	4+2+1 = 7	Valid.

From the given table 5 Spurious hits out of 9 windows. Hence need to calculate with the Rolling Hash for Calculation.

Spurious Condition ఉండదు

20) Rolling - Hash - Function :-

Deep
~~Decimal~~|depth

$$\boxed{\text{New Hash} = (\text{old hash} - \text{Outgoing}) \times \overset{\text{depth}}{d} + \text{incoming}}$$

- Remove the leftmost character's weighted - Value.
- shift left by multiplying d. ∵ d=10 → same for all.
- Add new character at Unit place

$$\text{dba} = \underline{\overset{d}{4}} \times 10^2 + \underline{\overset{b}{2}} \times 10^1 + \underline{\overset{a}{1}} \times 10^0$$ ⊢ According to its unit position → depth.

$$= 400 + 20 + 1$$

$$\boxed{\text{dba} = 421}$$ → Hash value of dba, | cca ⇒ $3\times10^2 + 3\times10^1 + 1\times10^0$ ⇒ 331

T= cca ccaa edba

shift	Window	Hash	Calculation.
0	cca ↳dba calculate చేసినట్టే చెయ్యాలి, old Hash లేదు కాబట్టి.	331	computed directly
1	cac	313	← $(331 - \overset{c\times10^2}{3\times100})\times10 + \overset{3\times10^0}{3}$
2	acc	133	← $(313 - \overset{c\times10^2}{3\times100})\times10 + \overset{3\times10^0}{3}$
3	cca	331	← $(133 - 1\times100)\times10+1$
4	caa	311	$(331 - 3\times100)\times10+1$
5	aae	115	$(311 - 3\times100)\times10+5$
6	aed	154	$(115 - 1\times100)\times10+4$
7	edb	542	$(154 - 1\times100)\times10+2$
8	dba	421	$(542 - 5\times100)\times10+1$ ↳ 421

Hence, Pattern dba is found Using Rolling Hash fn with hash 421 at index 9

Time complexity : O(n+m)

Space Complexity : O(n)

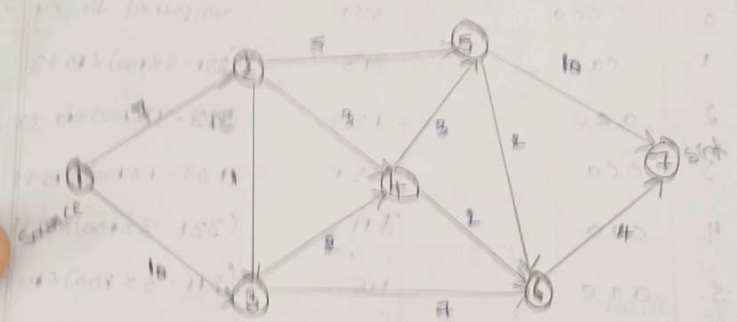
[illegible]

Network Flow Algorithm

1) Ford - Fulkerson - Algorithm :

The Ford Fulkerson is a greedy method used to compute the maximum possible flow from a source vertex to a sink vertex in a directed weighted flow-network.

Question :
Find the maximum flow through the given network using Ford Fulkerson Algorithm.



Source : The Source Vertex have all the outward edge no inward edge.

Sink : Sink will have all the inward edge, no outward edge.

Bottleneck Capacity : Bottleneck capacity of the path is the minimum capacity of any edge on the path. Ex: $① \xrightarrow{7} ② \xrightarrow{5} ⑤ \xrightarrow{10} ⑦ = 5$

[illegible] 7, 10 [illegible] flow [illegible]

Flow ÷ It is the total ~~path~~ value flown on a given path with respect to capacity. 5/10 [illegible] Total capacity

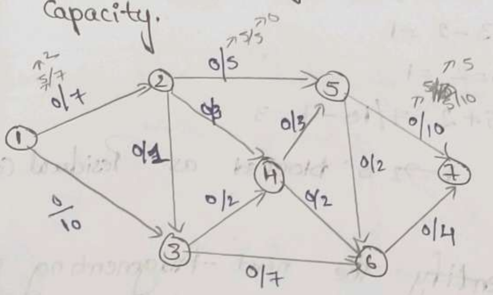
Augmenting Path ÷ It is the path calculated from Source to sink. It can be more than one.

Residual Capacity ÷ Each edge of a residual graph has a value called capacity, which is equal to Original capacity minus current flow

$\boxed{\text{capacity} - \text{flow}} = \boxed{5-1} = \boxed{4}$ ↓ Residual capacity.

Solⁿ ÷

Step I ÷ Initially Assign all flow as a '0' out of the capacity.



Step 2 ÷ Identify the Augmented path & Bottleneck.

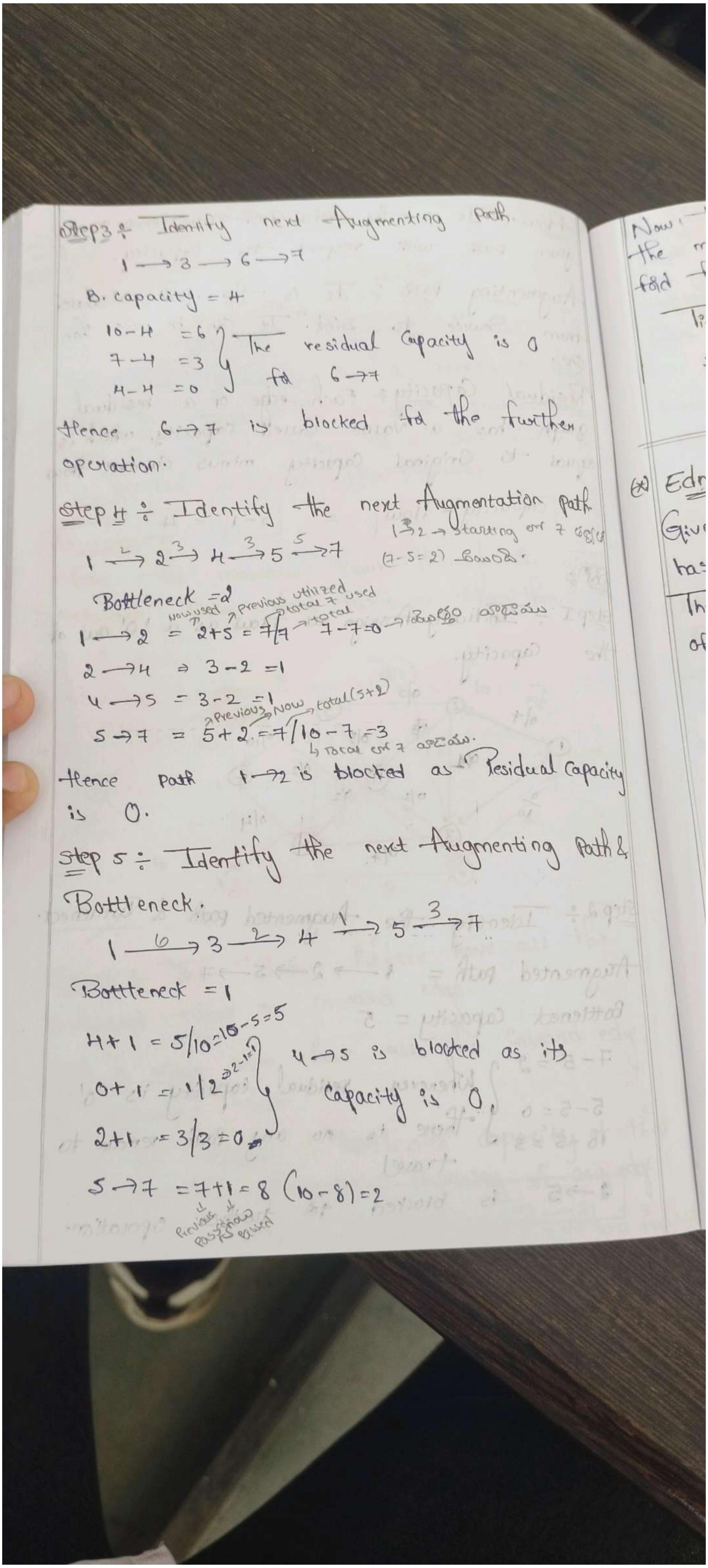
Augmented path = $1 \xrightarrow{7} 2 \xrightarrow{5} 5 \xrightarrow{10} 7$

Bottleneck capacity = 5

$7-5=2$
$5-5=0$
$10-5=5$

} Whenever residual capacity is '0' there is no any path remain to travel

$\boxed{2 \rightarrow 5 \text{ is blocked for further operation}}$



Step 3 : Identify next Augmenting path

$1 \longrightarrow 3 \longrightarrow 6 \longrightarrow 7$

B. capacity = 4

$10-4 = 6$
$7-4 = 3$
$4-4 = 0$
} The residual Capacity is 0 for $6 \rightarrow 7$

Hence $6 \rightarrow 7$ is blocked for the further operation.

Step 4 : Identify the next Augmentation path

$1 \xrightarrow{2} 2 \xrightarrow{3} 4 \xrightarrow{3} 5 \xrightarrow{5} 7$

$1 \xrightarrow{7} 2 \rightarrow$ starting లో 7 ఉండేది (7-5=2) ఉంటది.

Bottleneck = 2

$1 \longrightarrow 2 = \overset{\text{Now used}}{2} + \overset{\text{previous utilized used}}{5} = 7/\overset{\text{total 7}}{7} \rightarrow \overset{\text{total}}{7} - 7 = 0 \rightarrow$ మొత్తం వాడాము

$2 \longrightarrow 4 \Rightarrow 3-2 = 1$

$4 \longrightarrow 5 = 3-2 = 1$

$5 \rightarrow 7 = \overset{\text{Previous}}{5} + \overset{\text{Now}}{2} = 7/\overset{\text{total}(5+2)}{10} - 7 = 3$
↳ Total లో 7 వాడాము.

Hence path $1 \rightarrow 2$ is blocked as Residual capacity is 0.

Step 5 : Identify the next Augmenting path & Bottleneck.

$1 \xrightarrow{6} 3 \xrightarrow{2} 4 \xrightarrow{1} 5 \xrightarrow{3} 7$

Bottleneck = 1

$4+1 = 5/10 = 10-5 = 5$
$0+1 = 1/2 \Rightarrow 2-1 = 1$
$2+1 = 3/3 = 0$
} $4 \rightarrow 5$ is blocked as its capacity is 0.

$5 \rightarrow 7 = \underset{\text{Previous}}{7} + \underset{\text{Present now used}}{1} = 8 \; (10-8) = 2$

Now
the
find

li

(*) Edr
Give
has
Th
of

Now, there is no further path to travel, hence the maximum flow through given network using ford fulkerson method. is = 5+4+2+1 ⇒ 12

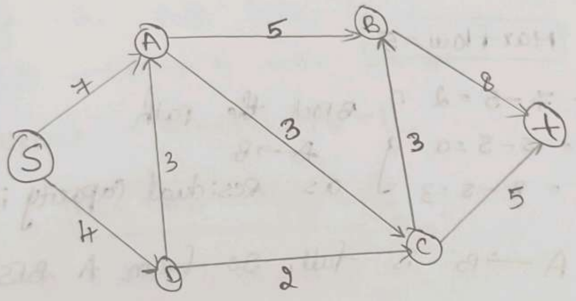
Time Complexity : $O(\overset{\text{edges}}{E} \times F) \rightarrow \text{Flow}$

Space Complexity : $O(\underset{\text{vertex}}{V} + \underset{\text{edge}}{E})$ (or) (V^2)

⊛ Edmonds – Karp Algorithm

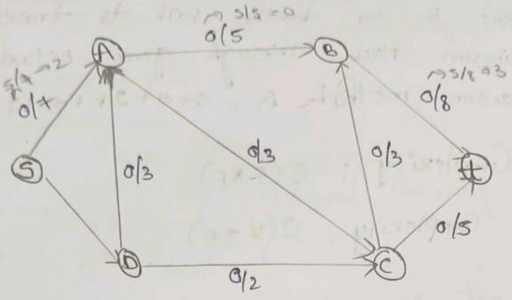
Given a directed graph where each edge has a capacity a source (S) & a sink 't' Then need to find the maximum amount of flow that can be sent from 'S' to 't'

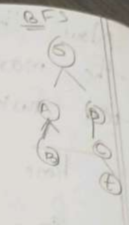
Edmonds Karp = Ford – fulkerson + $\overset{\text{Breadth 1st Search.}}{\text{BFS}}$



Solⁿ ÷

Step 1 ÷ Given Source S, Sink t, put all the flow from Source to Sink as a '0', out of total capacity.





Maxflow = Total flow Passed.

Step 2 : Repeat BFS till it is Unreached.

BFS levels :

$S \longrightarrow \{A, D\} \longrightarrow \{B, C\} \longrightarrow t$

Path to be ~~calculated~~ Consider.

$S \longrightarrow A \longrightarrow B \longrightarrow t$

Bottleneck = min(7, 5, 8) = 5

$\boxed{\text{Max flow} = 5}$

$S \to A = 7 - 5 = 2$
$A \to B = 5 - 5 = 0$
$B \to t = 8 - 5 = 3$

} Block the path $A \to B$ as Residual capacity is 0

Step 3 : $A \to B$ is full, so from A BFS goes to C

Consider path $S \xrightarrow{2} A \xrightarrow{3} C \xrightarrow{5} t$

Bottleneck = min (2, 3, 5) = 2

$\boxed{\text{Max flow} = 5 + 2 = 7}$

$S \to A = 5 + 2 = 7 = 7/7 = 7 - 7 = 0$
$A \to C = 3 - 2 = 1$
$C \to t = 5 - 2 = 3$

from the above path $S \rightarrow A$ is blocked hence close path $S \rightarrow A$ & identify BFS from A.

Step 4: $S \rightarrow A$ is full, so BFS must start from D.

$$S \rightarrow D \rightarrow C \rightarrow t$$

Bottleneck $= \min(4, 2, 3) = 2$

Max flow $= 7 + 2 = 9$

Previous B & [illegible] Path's close [illegible]

$S \rightarrow D = 4 - 2 = 2$

$D \rightarrow C = 2 - 2 = 0$

$C \rightarrow t = 2 + 2 = 4 \ (5 - 4) = 1$

Now close the path from $D \rightarrow C$ & identify the BFS equivalent from C.

→ from specific B there is no path travelling from Source to Sink.

Hence the path left is

$$S \rightarrow D \rightarrow A \rightarrow C \rightarrow t$$

Bottleneck $= \min(2, 3, 1, 1) = 1$

$$\boxed{\text{Max flow} = 9 + 1 = 10}$$

$S \rightarrow D = 2 + 1 = 3 \ (4 - 3) = 1$

$D \rightarrow A = 3 - 1 = 2$

$A \rightarrow C = 2 + 1 = 3 \ (3 - 3) = 0$

$C \rightarrow t = 4 + 1 = 5 \ (5 - 5) = 0$

Now no Augmenting path is remaining, Hence using Edmonds Karp method including Step 2, Step 3, Step 4 & Step 5 iterations the max flow from Source to Sink is 10.